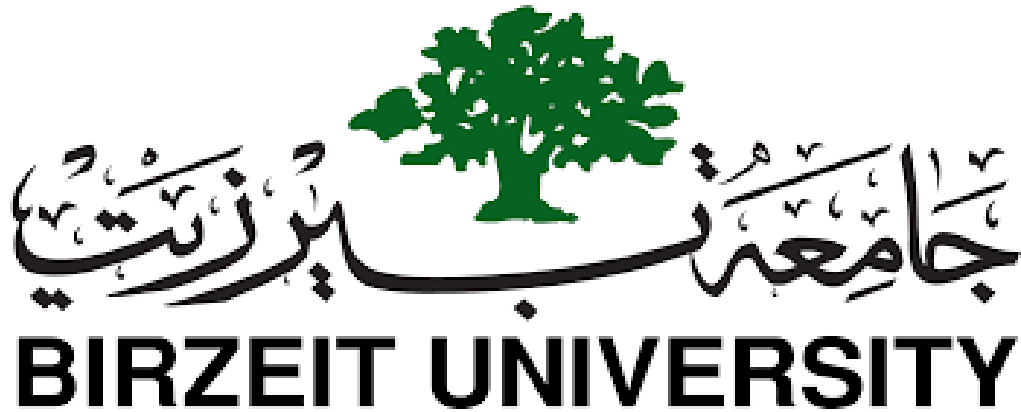




**Faculty of Engineering & Technology Electrical & Computer
Engineering Department**

ENCS4320

Project1

A Secure Communication Application

Prepared by:

Name: Yasmin badwan

ID: 1230271

Section:1

Name: Safaa Shalash

ID: 1230281

Section:1

Supervised by: Dr. Mohammed Hussein

Date: 26/6/2026.

Contents

System Architecture.....	1
Implemented Cryptographic Primitives.....	1
SHA-256.....	1
HMAC.....	1
HKDF.....	1
ChaCha20.....	1
Poly1305.....	2
ChaCha20-Poly1305 AEAD.....	2
X25519.....	2
Handshake Design.....	3
Key Schedule / Key Derivation.....	4
Message Format.....	5
Nonce Construction.....	6
Replay and Tampering Protection.....	6
Testing.....	7
1. Unit Testing of Cryptographic Primitives.....	7
2. Protocol-Level Testing.....	7
3. Security and Adversarial Testing.....	8
Security Discussion.....	9
Limitations.....	9
AI Usage Statement.....	10
Individual Contribution Statement.....	10
Conclusion.....	11

System Architecture

Implemented Cryptographic Primitives

In this project, all required cryptographic primitives were implemented from scratch in pure Python according to their official specifications. These primitives form the foundation of the SecureChannel protocol because they are used for hashing, authentication, key derivation, encryption, message integrity, and secure key exchange.

SHA-256

SHA-256 was implemented according to the NIST FIPS 180-4 specification. Its input is any arbitrary-length message, such as handshake data or intermediate cryptographic values. Internally, the message is padded, divided into 512-bit blocks, and processed through compression rounds using bitwise operations and modular addition. The output is a fixed 256-bit hash digest. In SecureChannel, SHA-256 is mainly used as the underlying hash function for HMAC-SHA-256 and HKDF, which are needed for handshake authentication and key derivation.

HMAC

HMAC was implemented according to RFC 2104 using SHA-256 as the underlying hash function. Its inputs are a secret key and a message, such as the pre-shared key and the handshake transcript. Internally, the key is combined with inner and outer padding values, and SHA-256 is applied twice to produce an authentication value. The output is a message authentication tag. In SecureChannel, HMAC is used to authenticate the handshake transcript and is also used internally by HKDF.

HKDF

HKDF was implemented according to RFC 5869 using HMAC-SHA-256. Its inputs are the X25519 shared secret, the pre-shared key used as salt, and context information. Internally, HKDF works in two stages: the extract step produces a pseudorandom key, and the expand step derives the required output key material. The output is independent session keys and nonce material. In SecureChannel, HKDF is used after the X25519 key exchange to derive separate keys for client-to-server and server-to-client communication.

ChaCha20

ChaCha20 was implemented according to RFC 8439. Its inputs are a 256-bit key, a nonce, a counter, and the data to be encrypted or decrypted. Internally, ChaCha20 generates a keystream using repeated quarter-round operations based on additions, XORs, and rotations. This keystream is XORed with the input data. The output is ciphertext during encryption or plaintext during decryption. In SecureChannel, ChaCha20 is used as the encryption component inside ChaCha20-Poly1305 AEAD.

Poly1305

Poly1305 was implemented according to RFC 8439. Its inputs are a one-time key and the data that needs to be authenticated. Internally, the data is divided into blocks and processed using modular arithmetic to compute an authentication value. The output is a 128-bit authentication tag. In SecureChannel, Poly1305 is used inside ChaCha20-Poly1305 AEAD to verify message integrity and authenticity, allowing the receiver to detect any modification to the ciphertext or associated data.

ChaCha20-Poly1305 AEAD

ChaCha20-Poly1305 AEAD was implemented according to RFC 8439 Section 2.8. Its inputs are a key, nonce, plaintext, and associated data. Internally, ChaCha20 encrypts the plaintext, while Poly1305 authenticates both the associated data and the ciphertext. The output is ciphertext together with an authentication tag. In SecureChannel, this construction protects all application messages after the handshake by providing confidentiality, integrity, and authenticity. The associated data includes the plaintext message header, such as the protocol version, message type, sender identity, and sequence number.

X25519

X25519 was implemented according to RFC 7748. Its inputs are a private key and the other party's public key. Internally, it performs elliptic-curve scalar multiplication using the Montgomery ladder. The output is a shared secret that both parties can compute independently. In SecureChannel, X25519 is used during the handshake phase to perform ephemeral Diffie-Hellman key exchange between the client and the server. The resulting shared secret is then passed to HKDF to derive the session keys.

Together, these primitives form the complete SecureChannel cryptographic workflow. X25519 establishes the shared secret, HMAC authenticates the handshake, HKDF derives the required

session keys and nonce material, and ChaCha20-Poly1305 AEAD protects the actual application messages during secure communication.

Handshake Design

The handshake phase is the first step in SecureChannel. Its goal is to allow the client and server to agree on a shared secret and authenticate each other before starting encrypted communication. Our handshake combines ephemeral X25519 key exchange, PSK-based HMAC authentication, and HKDF key derivation.

First, both the client and the server generate fresh ephemeral X25519 key pairs for the current session. The client sends its public key to the server, and the server sends its public key back to the client. These public keys are allowed to travel over the network because only the private keys must remain secret.

After the public keys are exchanged, each side computes the same X25519 shared secret. The client uses its private key with the server's public key, while the server uses its private key with the client's public key. This gives both parties the same shared secret without ever sending it directly over the network.

Then, both sides build the same handshake transcript. The transcript contains the protocol version, the client public key, the server public key, the client identity, and the server identity. This makes the transcript describe the exact session that both sides are agreeing on.

To authenticate the handshake, each side computes an HMAC-SHA-256 tag using the pre-shared key. In our implementation, the HMAC input is the handshake transcript together with the shared secret. The client sends its tag to the server, and the server verifies it. Then the server sends its tag back to the client, and the client verifies it. If any tag does not match, the connection is aborted immediately.

This prevents man-in-the-middle attacks because an attacker may intercept or replace public keys, but they do not know the pre-shared key. Without the PSK, the attacker cannot produce a valid HMAC tag for the transcript. Also, since the transcript includes both public keys, identities, and the protocol version, any modification to the handshake data changes the HMAC result and causes verification to fail.

Once the handshake is successfully authenticated, both sides use HKDF to derive the session material from the shared secret. The derived output is split into two encryption keys and two nonce bases: one for client-to-server messages and one for server-to-client messages. These values are then used by the AEAD layer to protect the secure chat messages

Key Schedule / Key Derivation

After the X25519 handshake is completed, both the client and the server have the same shared secret. This shared secret is not used directly as an encryption key. Instead, it is passed into HKDF to derive the required session keys and nonce material in a secure and structured way.

In our implementation, HKDF uses the X25519 shared secret as the input key material. The pre-shared key is used as the HKDF salt, which mixes the long-term shared secret into the key derivation process. HKDF is based on HMAC-SHA-256 and follows two steps: extract and expand. The extract step produces a pseudorandom key from the shared secret and PSK, while the expand step generates the final session material using the context string SecureChannel-X25519.

Both parties run the same HKDF call:

```
keys = hkdf(shared, PSK, b"SecureChannel-X25519", 88)
```

The HKDF output length is 88 bytes because the protocol requires two 32-byte AEAD encryption keys and two 12-byte nonce bases:

- 32 bytes client-to-server encryption key
- 32 bytes server-to-client encryption key
- 12 bytes client-to-server nonce base
- 12 bytes server-to-client nonce base

Since the client and server use the same X25519 shared secret, the same PSK, and the same context string, they both derive the same 88 bytes of key material. This output is then split into four values: a client-to-server encryption key, a server-to-client encryption key, a client-to-server nonce base, and a server-to-client nonce base.

The first 32 bytes are used as the client-to-server encryption key, and the next 32 bytes are used as the server-to-client encryption key. The following 12 bytes are used as the client-to-server nonce base, and the final 12 bytes are used as the server-to-client nonce base.

Separate keys and nonce bases are used for each communication direction. This means that messages sent from the client to the server use different cryptographic material than messages sent

from the server to the client. This separation makes the two directions independent and avoids using the same key and nonce material for both sides of the secure channel.

Message Format

After the handshake is completed, the client and server exchange messages using a structured frame format. Each frame contains a plaintext header and a message payload. The header is used as associated data, which means it is not encrypted, but it is authenticated by ChaCha20-Poly1305 AEAD.

The frame header is 9 bytes long. It consists of three fields: `sender_id`, `seq`, and `msg_type`. The `sender_id` field is 4 bytes and identifies whether the message was sent by the client or the server. The `seq` field is also 4 bytes and represents the message sequence number, which is used for replay protection. The `msg_type` field is 1 byte and identifies the message type, such as a normal chat message or a termination message.

The frame format is:

|sender_id (4 bytes) |seq (4 bytes)|msg_type (1 byte) |payload (variable length)|

In our implementation, the first 9 bytes of the encoded frame are used as associated data:

```
aad = raw[:9]
```

The payload contains the actual chat message and is encrypted using ChaCha20-Poly1305 AEAD. The header remains visible so the receiver can read the sender identity, sequence number, and message type, but any modification to the header will cause authentication verification to fail.

When the message is sent over TCP, it is wrapped in a simple wire format that includes the associated data length, the associated data, the ciphertext length, the ciphertext, and the authentication tag.

The transmitted format is:

|aad_len|aad |ciphertext_len|ciphertext|tag|

This message format allows the receiver to parse the incoming data, verify that the header was not modified, decrypt the ciphertext, and reject any tampered message.

Nonce Construction

ChaCha20-Poly1305 requires a unique 12-byte nonce for every encrypted message under the same encryption key. Reusing the same nonce with the same key is unsafe, so our protocol constructs a different nonce for each message.

During key derivation, HKDF produces two nonce bases: one for client-to-server messages and one for server-to-client messages. Each direction has its own nonce base, which keeps the two communication directions separated.

For every message, the sender uses the current sequence number to create the final nonce. The sequence number is converted into a 12-byte little-endian counter, then XORed with the direction-specific nonce base.

The nonce construction is:

```
counter_bytes = counter.to_bytes(12, "little")
nonce = bytes(a ^ b for a, b in zip(nonce_base, counter_bytes))
```

Since the sequence number increases for every message, the final nonce changes for every encryption operation. This prevents nonce reuse during a session. Also, because client-to-server and server-to-client messages use separate keys and nonce bases, the two directions do not share the same key/nonce material.

Replay and Tampering Protection

The system protects against replay attacks using sequence numbers and against message modification using authenticated encryption.

Each message contains a sequence number (seq) in its header. Both the client and server maintain an expected sequence counter for each direction. When a message is received, its sequence number

is checked against the expected value. If a message is repeated, delayed, or received out of order, it is rejected. This prevents attackers from replaying previously captured messages.

In addition to replay protection, message integrity is ensured using ChaCha20-Poly1305 AEAD. The authentication tag covers both the ciphertext and the associated data (AAD), which includes the message header. This means that any modification to the sender ID, sequence number, message type, or ciphertext will cause authentication to fail.

If the authentication tag verification fails, the message is immediately discarded and not processed. This ensures both integrity and authenticity of all communication between client and server

Testing

The SecureChannel implementation was tested using a combination of unit tests and system-level tests to ensure correctness and security of all components.

1. Unit Testing of Cryptographic Primitives

Each cryptographic module was tested independently using dedicated test files included in the project. These tests verify correctness against known values and expected behavior:

- SHA-256: tested using `test_sha256.py`
- HMAC-SHA-256: tested using `test_hmac.py`
- HKDF: tested using `test_hkdf.py`
- ChaCha20: tested using `test_chacha20.py`
- Poly1305: tested using `test_poly1305.py`
- ChaCha20-Poly1305 AEAD: tested using `test_aead.py`
- X25519: tested using `test_x25519.py`

These tests ensure that each cryptographic primitive produces correct outputs according to its specification.

2. Protocol-Level Testing

Integration tests were performed using the full client-server system (`client.py` and `server.py`). These tests verify:

- Successful handshake between client and server
- Matching shared secret and derived session keys
- Successful encrypted message exchange using ChaCha20-Poly1305 AEAD

3. Security and Adversarial Testing

Additional tests were performed to verify security properties:

- Replay attack simulation using `test_replay.py` confirms that repeated or out-of-order messages are rejected
- Message modification tests confirm that any change in ciphertext or header causes authentication failure
- Invalid handshake scenarios confirm that incorrect authentication leads to immediate connection termination

Overall, the testing confirms that both the cryptographic primitives and the SecureChannel protocol function correctly and securely.

Security Discussion

The SecureChannel protocol is designed to provide confidentiality, integrity, and authentication using well-established cryptographic primitives.

Confidentiality is achieved using ChaCha20, which ensures that all application messages are encrypted using a secure stream cipher. Integrity and authenticity are provided using the ChaCha20-Poly1305 AEAD construction, which ensures that any modification to ciphertext or associated data is detected during verification.

Authentication during the handshake is achieved using HMAC-SHA-256 with a pre-shared key. This ensures that only parties who know the PSK can successfully complete the handshake and establish a session. The inclusion of both public keys, identities, and protocol version in the handshake transcript binds the session context and prevents impersonation attacks.

Key agreement is performed using X25519 Diffie-Hellman, which allows both parties to derive the same shared secret without transmitting it over the network. This ensures forward secrecy since ephemeral keys are used for each session.

HKDF is used to derive cryptographic keys from the shared secret in a secure and structured way. It produces separate keys for each communication direction, ensuring that client-to-server and server-to-client traffic are cryptographically independent.

Replay protection is achieved using sequence numbers. Each message is assigned a unique sequence number, and any repeated or out-of-order messages are rejected. This prevents attackers from reusing previously captured messages.

Overall, the combination of these mechanisms ensures that the protocol is resistant to common attacks such as eavesdropping, tampering, replay attacks, and man-in-the-middle attacks.

Limitations

Although SecureChannel provides strong cryptographic guarantees, it is designed as an educational project and has some limitations.

First, the system is implemented as a command-line application and does not include a graphical user interface. This makes it less user-friendly compared to production-ready messaging systems. Second, the protocol relies on a pre-shared key (PSK) that must be securely exchanged out-of-band before communication begins. If the PSK is compromised, the security of the handshake is affected.

Third, the implementation is intended for local testing using TCP sockets on localhost. While it can be extended to network environments, it has not been fully tested in large-scale or real-world distributed systems.

Finally, the system focuses on demonstrating cryptographic principles rather than providing a fully optimized or production-level secure messaging platform.

AI Usage Statement

AI tools were used only as a learning and clarification aid during the project. They helped us better understand the cryptographic concepts, specifications, and design choices by explaining ideas from the project references and related online resources in a clearer way. The tools were used to support our understanding of why each primitive is needed, how it fits into the SecureChannel protocol, and how the different parts of the system work together. They were not used to replace our implementation work or final design decisions. All code, testing, integration, and final verification were completed and reviewed by the team members.

Individual Contribution Statement

Yasmin Badwan

Yasmin was responsible for the key establishment, authentication, and session setup components of the project. She implemented the X25519 key exchange module (`x25519.py`), including Montgomery ladder scalar multiplication and key clamping. She also implemented the HMAC module (`hmac.py`) and the HKDF key derivation module (`hkdf.py`), which are used to authenticate the handshake and derive the session keys and nonce material from the shared secret. In addition, Yasmin implemented the full handshake logic in `Handshake.py` and integrated the cryptographic components into the client-server communication system by writing `client.py` and `server.py`, including TCP socket handling, handshake protocol flow, and AEAD send/receive functions.

Safaa Shalash

Safaa was responsible for the encryption, hashing, message authentication, and replay protection components of the project. She implemented the SHA-256 hash function (`sha256.py`) according to the FIPS 180-4 specification, the ChaCha20 stream cipher (`chacha20.py`), the Poly1305 message authentication code (`Poly1305.py`), and the ChaCha20-Poly1305 AEAD construction (`aead.py`). She also implemented the replay protection module (`replay_protection.py`) and the protocol frame structure (`protocol_frames.py`), which define the message format and use sequence numbers to detect and reject replayed or out-of-order messages.

Both team members participated in integrating the complete SecureChannel protocol, testing the client-server communication, debugging implementation issues, reviewing the final report, and preparing for the oral discussion.

Conclusion

In this project, we designed and implemented SecureChannel, a secure communication application that allows a client and server to exchange protected messages over TCP. The project combined several cryptographic primitives into one complete protocol, including X25519 for key exchange, HMAC for handshake authentication, HKDF for key derivation, and ChaCha20-Poly1305 AEAD for encrypted and authenticated messaging.

The system successfully establishes a shared secret between the two parties, authenticates the handshake using a pre-shared key, derives separate session keys and nonce bases for each communication direction, and protects all application messages using authenticated encryption. The use of associated data allows the protocol to authenticate plaintext headers such as sender identity, message type, and sequence number without encrypting them.

Replay and tampering protection were also included in the design. Sequence numbers are used to detect repeated or out-of-order messages, while the AEAD authentication tag ensures that any modification to the ciphertext or header is detected before the message is accepted. Testing was performed at both the primitive level and the protocol level to verify correctness and security behavior.

Overall, SecureChannel demonstrates how modern cryptographic primitives can be implemented and combined to build a secure communication protocol. Although the application is educational and not intended for production use, it provides a clear practical understanding of secure key exchange, authentication, key derivation, authenticated encryption, and replay protection.

